# Natural Language Processing 2

## Retrieval-Augmented Generation (RAG)

Fatemeh Dehrouyeh



# Contents


1. Retrieval-Augmented Generation (RAG)
2. Part 1: Retrieval Methods
3. Part 2: Query Strategies
4. Part 3: Reasoning Patterns
5. Part 4: Knowledge Structures
6. Part 5: Design Choices
7. Part 6: Evaluation

# Current Section

1 Retrieval-Augmented Generation (RAG)

2 Part 1: Retrieval Methods

3 Part 2: Query Strategies

4 Part 3: Reasoning Patterns

5 Part 4: Knowledge Structures

6 Part 5: Design Choices

7 Part 6: Evaluation

# What is RAG?

**RAG** stands for **Retrieval-Augmented Generation**.

It combines two steps:

- **Retrieval:** find relevant information from an external knowledge source.
- **Generation:** use that information to generate the final answer.

**Main idea:**
Instead of relying only on the model's internal memory, RAG gives the model relevant documents before it answers.
**RAG augments the LLM's prompt with relevant information retrieved from external sources before generating an answer.**

$$\text{User question} + \text{retrieved evidence} \rightarrow \text{LLM answer}$$

# Why Do We Need RAG?

LLMs have limitations:

- Their training data can be outdated.
- They may hallucinate unsupported facts.
- They may not know private or domain-specific documents.
- They cannot always cite where an answer came from.

RAG helps by connecting the model to external knowledge.

**Examples of external knowledge:**

- PDFs
- company documents
- websites
- research papers
- databases

# Examples of RAG

Retrieval-Augmented Generation can be applied in different ways depending on how the external information is retrieved and used:

- **Document Retrieval:** Attaching a document (e.g., PDF, Word file) to the chat, so the model can retrieve and use its content.
- **Knowledge Base Retrieval:** Searching a structured database or knowledge base (e.g., FAQs, company manuals) to answer user questions.
- **Web Retrieval:** Querying the internet or an enterprise search system for the most up-to-date facts and news.
- **Hybrid Approaches:** Combining multiple retrieval sources (documents + web + databases) before generating the final answer.

# Three Main Steps of RAG

1. **Retrieve relevant information**
   Search an external knowledge base to find information relevant to the user's question.
   In vector RAG, retrieval uses similarity between question and document embeddings.

2. **Augment the prompt**
   Combine the original user question with the retrieved information to create an augmented prompt.

3. **Generate a response**
   Pass the augmented prompt to the LLM. The model uses the retrieved context to generate a relevant, evidence-supported answer.

Retrieve $\longrightarrow$ Augment $\longrightarrow$ Generate

# RAG Pipeline

A typical RAG system has five main steps:

1. Split documents into smaller chunks.
2. Convert each chunk into an embedding.
3. Store the embeddings in a vector database.
4. Retrieve the most relevant chunks for a user question.
5. Give the retrieved chunks to the LLM to generate an answer.

Documents → Chunks → Embeddings → Retriever → LLM Answer

# Preparing the RAG Knowledge Base

Before retrieval, we prepare the documents so that the RAG system can search them efficiently.

1. **Collect documents:** Gather information from PDFs, websites, reports, or other sources.
2. **Chunk documents:** Divide documents into smaller pieces called chunks.
3. **Create embeddings:** Convert each chunk into a numerical vector using an embedding model.
4. **Store and index:** Store the chunks, their embeddings, and metadata in a searchable system.

$$\text{Documents} \rightarrow \text{Chunks} \rightarrow \text{Embeddings} \rightarrow \text{Vector Database}$$

**Important**

This is the preparation process for vector-based RAG. Other retrieval methods may not require embeddings.

# Step 1: Chunking

Long documents are usually split into smaller pieces called **chunks**.

**Why?**

- LLMs have limited context length.
- Retrieval works better with focused pieces of text.
- A whole document may contain many unrelated topics.

**Example:**

$$\text{10-page PDF} \rightarrow \text{many smaller chunks}$$

A chunk may be:

- one paragraph
- a few sentences
- 300 to 800 tokens

# Step 2: Embeddings

An **embedding** is a numerical vector that represents the meaning of text.

Texts with similar meanings should have similar embeddings.

**Example:**

"car" $\approx$ "automobile"
"doctor" $\approx$ "physician"

In RAG, we create embeddings for:

- document chunks
- user questions

**Embedding Model:**
A pretrained embedding model converts each document chunk into a numerical vector that captures its meaning.
The same embedding model is typically used to encode the user's question for similarity search.

**Embedding Model**

A pretrained encoder-only model, such as Sentence-BERT, can generate embeddings.
The query and document chunks are encoded independently using a bi-encoder architecture.

# Step 3: Vector Database

A **vector database** stores embeddings and allows similarity search.

When the user asks a question, the system converts the question into an embedding.
Then it compares the question embedding with document chunk embeddings using a similarity metric.

A common choice is **cosine similarity**:

$$\cos(\theta) = \frac{\vec{q} \cdot \vec{d}}{\|\vec{q}\|\|\vec{d}\|}$$

where:

- $\vec{q}$ is the question embedding.
- $\vec{d}$ is a document chunk embedding.

The vector database returns the chunks with the highest similarity scores.

$$\text{Question embedding} \rightarrow \text{most similar chunk embeddings}$$

# Step 4: Top-$k$ Retrieval

RAG usually retrieves the top-$k$ most relevant chunks.

**Example:**

$$k = 3$$

The system returns the 3 chunks most similar to the question.

**If $k$ is too small:**

- important evidence may be missing.

**If $k$ is too large:**

- irrelevant text may confuse the model.

# Step 5: Generation

After retrieval, the retrieved chunks are inserted into the prompt.

**Prompt structure:**

$$\text{Question} + \text{Retrieved Context} \rightarrow \text{Answer}$$

**Example:**

> *Use the following context to answer the question. If the answer is not in the context, say you do not know.*

This helps the model answer using evidence instead of only relying on memory.

# RAG Example

**User question:**
"What is the refund policy?"

**Retrieved chunk:**
"Customers can request a refund within 30 days of purchase if the item is unused."

**Generated answer:**
"You can request a refund within 30 days, as long as the item is unused."

**Key point:**
The answer is grounded in the retrieved document.

# RAG vs. Standard LLM

**Standard LLM:**

- Answers from internal model knowledge.
- May not know recent or private information.
- May hallucinate when uncertain.

**RAG system:**

- Retrieves external evidence first.
- Uses that evidence to answer.
- Can cite or trace the source.

$$\text{LLM only} = \text{memory-based answer}$$

$$\text{RAG} = \text{evidence-based answer}$$

# Where RAG Can Fail

RAG can still make mistakes.

Common failure points:

- The retriever finds the wrong chunks.
- The correct chunk is missing from the database.
- The chunks are too short or too long.
- The LLM ignores the retrieved evidence.
- The LLM combines evidence incorrectly.

**Important idea:**
RAG quality depends on both retrieval quality and generation quality.

# Retrieval Failure Example

**User question:**
"What is the return deadline for damaged products?"

**Wrong retrieved chunk:**
"Customers can return unused products within 30 days."

**Missing correct chunk:**
"Damaged products must be reported within 7 days of delivery."

**Problem:**
The LLM may answer "30 days," even though the correct answer is "7 days."

**Takeaway:**
If retrieval fails, generation will likely fail too.

# Hallucination in RAG

RAG reduces hallucination, but it does not eliminate it.

The model may still:

- add information not found in the retrieved context
- overgeneralize from one chunk
- answer even when the evidence is missing
- mix unrelated chunks together

A safer instruction is:

> *Answer only using the retrieved context. If the answer is not in the context, say you do not know.*

# Evaluating RAG

RAG systems should be evaluated in two parts.

1. **Retrieval evaluation:**
   - Did the system retrieve the right evidence?
   - Was the correct chunk in the top-$k$ results?

2. **Generation evaluation:**
   - Is the answer correct?
   - Is the answer grounded in the retrieved evidence?
   - Does the answer avoid unsupported claims?

**Main idea:**
A good RAG answer needs both good retrieval and good generation.

# Common RAG Evaluation Metrics

**Retrieval metrics:**

- Recall@k
- Precision@k
- Mean Reciprocal Rank (MRR)
- Normalized Discounted Cumulative Gain (NDCG@k)

**Generation metrics:**

- Answer correctness
- Faithfulness or groundedness
- Citation accuracy

# Retrieval Metric: Recall@k

**Main question:** Did we retrieve most of the relevant documents?

$$Recall@k = \frac{\text{number of relevant documents retrieved in top } k}{\text{total number of relevant documents}}$$

**Example:**

- A database contains 1,000 documents.
- Exactly 5 documents are relevant to the query.
- The retriever ranks documents by estimated relevance.
- We examine the top 10 results ($k = 10$):
  - 4 are genuinely relevant.
  - 6 are irrelevant.
  - 1 relevant document ranks below the top 10.

$$Recall@10 = \frac{4}{5} = 0.8 = 80\%$$

**Why it matters in RAG**

Higher recall means fewer relevant documents are missed.

# Retrieval Metric: Precision@k

**Main question:** Of the retrieved documents, how many are actually relevant?

$$\textit{Precision@}k = \frac{\text{number of relevant documents retrieved in top } k}{k}$$

**Example:**

- Retrieved documents: top 10
- Relevant documents in top 10: 4
- Irrelevant documents in top 10: 6

$$\textit{Precision@}10 = \frac{4}{10} = 0.4$$

**Why it matters in RAG**

High precision means the LLM receives less irrelevant context.

# Retrieval Metrics: Precision vs. Recall

**Main question:** What happens when we increase $k$?

**Example:** Suppose a database contains 1,000 documents, of which only 5 are relevant.

| $k$ | Relevant Found | Precision@k | Recall@k |
|---|---|---|---|
| 3 | 2 | 66.7% | 40% |
| 5 | 3 | 60% | 60% |
| 10 | 4 | 40% | 80% |
| 20 | 5 | 25% | 100% |

**Observation:**

- As $k$ increases, recall increases because more relevant documents are found.
- Precision decreases because more irrelevant documents are included.
- The retriever's ranking does not change; only the number of examined results changes.

**Why it matters in RAG**

A smaller $k$ provides less context, often with higher precision. A larger $k$ improves recall but may introduce irrelevant information.

# Retrieval Metric: Mean Reciprocal Rank, MRR

**Main question:** How high is the first relevant document ranked?

$$\textit{Reciprocal Rank} = \frac{1}{\text{rank of first relevant result}}$$

$$MRR = \frac{1}{N}\sum_{i=1}^{N}\frac{1}{\text{rank}_i}$$

**Example with 3 questions:**

| Question | First relevant rank | Reciprocal rank |
|---|---|---|
| 1 | 1 | 1.0 |
| 2 | 2 | 0.5 |
| 3 | 5 | 0.2 |

$$MRR = \frac{1.0 + 0.5 + 0.2}{3} = 0.567$$

**Why it matters in RAG**

High MRR means useful evidence appears near the top of the retrieved results.

# Retrieval Metric: NDCG@k

**Main question:** Are relevant chunks ranked near the top?

**Normalized Discounted Cumulative Gain (NDCG)** evaluates the quality of a ranked list of retrieved chunks.

$$DCG@k = \sum_{i=1}^{k} \frac{rel_i}{\log_2(i+1)}$$

$$NDCG@k = \frac{DCG@k}{IDCG@k}$$

where $rel_i \in \{0, 1\}$ indicates whether the chunk at rank $i$ is relevant.

- **Cumulative:** Considers all chunks in the top $k$ ranks.
- **Discounted:** Relevant chunks at higher ranks contribute more.
- **Normalized:** Divides by the ideal DCG, obtained from the best possible ranking.

**Why it matters in RAG**

NDCG@k rewards systems that rank relevant evidence closer to the top. A score of 1 indicates an ideal ranking.

# How Do We Evaluate RAG Retrieval Metrics?

**Main question:** Where do we get the data to calculate Precision@k, Recall@k, MRR, and NDCG@k?

**Public benchmark datasets:**

- **MS MARCO:** Millions of text passages, queries, and relevance labels.
- **BEIR:** Retrieval benchmarks across scientific, medical, and general domains.
- **TREC:** Human-assessed relevance judgments and retrieval evaluation datasets.

**How evaluation works:**

1. The benchmark provides queries, documents (or chunks), and ground-truth relevance labels.
2. Our retriever ranks the documents for each query.
3. We compare the rankings against the labels.
4. We calculate:
   - **Precision@k:** How many of the top $k$ retrieved documents are relevant?
   - **Recall@k:** How many known relevant documents appear in the top $k$?
   - **MRR:** How early does the first relevant document appear, on average?

**Datasets:**

MS MARCO BEIR TREC Deep Learning 2022

# RAG Key Takeaways

- RAG combines retrieval with generation.
- It gives the LLM external evidence before answering.
- It is useful for recent, private, or domain-specific knowledge.
- Its quality depends heavily on the retrieved chunks.
- RAG reduces hallucination, but it does not remove it completely.

**RAG turns an LLM from a memory-only system into an evidence-supported system.**

# Moving Beyond Basic RAG

**Learning Objectives**

- Understand what basic RAG does and why it may not be enough.
- Compare retrieval methods: keyword, vector, hybrid, and reranked retrieval.
- Explain query strategies such as rewriting, expansion, multi-query retrieval, HyDE, and contextual retrieval..
- Identify advanced RAG patterns: multi-hop, corrective, self-reflective, and agentic RAG.
- Explore knowledge structures such as graph, hierarchical, structured-data, and multimodal RAG.
- Evaluate RAG systems using metrics beyond answer accuracy.

# What We Have Learned So Far

Our current RAG pipeline is **naive RAG**, also called **basic vector RAG**.




**Main Point**

Basic RAG retrieves chunks. Advanced RAG improves what to retrieve, how to retrieve, how to verify, and how to reason across evidence.

# Why Basic RAG Is Only the Surface

Basic RAG often assumes:

- One user question is enough.
- One retrieval call is enough.
- Semantic similarity is enough.
- Top-$k$ chunks are reliable.
- Text chunks are the main knowledge unit.
- The LLM will use the evidence correctly.

Real systems break these assumptions.

**Production RAG question**

How do we retrieve the **right evidence**, from the **right source**, at the **right level of detail**, with **verification**?

# RAG Taxonomy

| Category | Examples |
|---|---|
| Retrieval method | Keyword, vector, hybrid, reranked |
| Query strategy | Rewriting, expansion, multi-query, HyDE, contextual retrieval |
| Reasoning pattern | Multi-hop, corrective, self-reflective, agentic |
| Knowledge structure | Hierarchical, graph, structured data, multimodal |
| Evaluation | Recall, precision, groundedness, citation accuracy, cost, latency |

# Current Section

1 Retrieval-Augmented Generation (RAG)

2 Part 1: Retrieval Methods

3 Part 2: Query Strategies

4 Part 3: Reasoning Patterns

5 Part 4: Knowledge Structures

6 Part 5: Design Choices

7 Part 6: Evaluation

# Types of Retrieval Methods in RAG

**How can a RAG system retrieve relevant information?**

- **Keyword Retrieval:** Uses BM25, TF-IDF, or exact keyword matching.
- **Vector Retrieval:** Uses embeddings and semantic similarity.
- **Hybrid Retrieval:** Combines keyword and vector retrieval.
- **Structured-Data Retrieval:** Uses SQL queries or APIs to access data.
- **Graph-Based Retrieval:** Uses entities and relationships to find information.

**Key Point**

Retrieval in RAG does not always require embeddings or vector databases. Different methods can be used depending on the data and the question.

# 1. Keyword RAG

**Keyword RAG** uses traditional search.

Common methods:

- BM25
- TF-IDF
- Exact keyword matching

**Best for**

Legal clauses, product codes, error messages, names, dates, section numbers, rare technical terms.

**Example:** "What does Section 12.4 say about termination?"

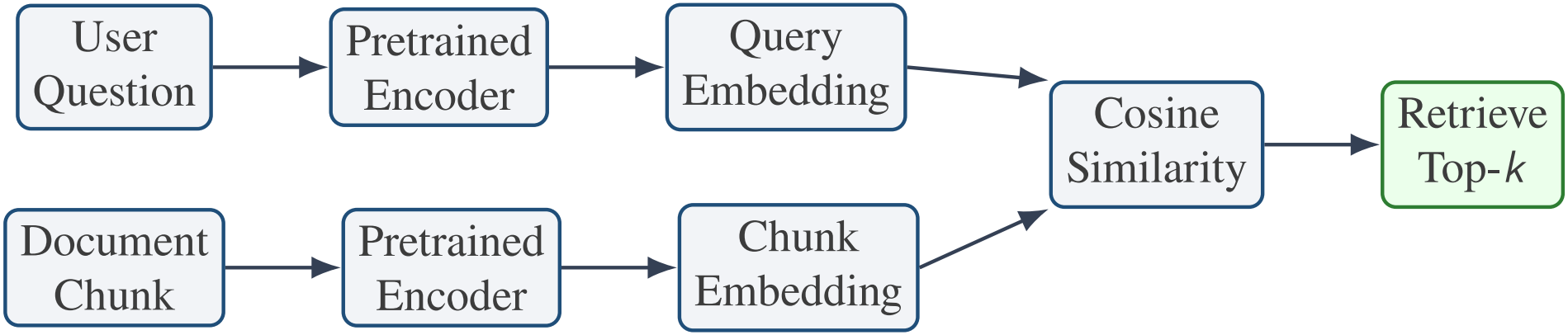
Vector search may miss "12.4". Keyword search preserves the exact token.

**How BM25 Works**

BM25 ranks documents based on keyword overlap with the query, considering term frequency, term rarity, and document length. BM25 takes the user query and compares it against each document or chunk independently, producing a relevance score for each pair. It does not require embeddings or semantic similarity.

# 2. Vector RAG

**Vector RAG** retrieves chunks using semantic similarity.




**Best for**

Questions where the wording differs from the document but the meaning is similar.

- A pretrained bi-encoder converts the query and document chunks into embeddings independently.
- Similarity is commonly measured using cosine similarity.
- Chunks with higher similarity scores are retrieved as relevant candidates.
- Exact keyword matching is not required.

**Example:** "car" can match "automobile", "doctor" can match "physician".

# Sentence-BERT (SBERT)

**Sentence-BERT** (Reimers & Gurevych, 2019) adapts pretrained BERT to produce sentence embeddings suitable for semantic similarity search.

- Uses a **bi-encoder architecture**: the query and document chunks are encoded independently.
- Uses a shared encoder to produce embeddings in a comparable vector space.
- At retrieval time, embeddings can be compared using **cosine similarity**.

$$\text{Query} \xrightarrow{\text{Encoder}} \mathsf{q}$$

$$\text{Chunk} \xrightarrow{\text{Encoder}} \mathsf{d}$$

$$\mathsf{Similarity}(\mathsf{q}, \mathsf{d}) = \frac{\mathsf{q} \cdot \mathsf{d}}{\|\mathsf{q}\| \|\mathsf{d}\|}$$

Source: Reimers & Gurevych (2019), Sentence-BERT.

# Keyword vs. Vector Retrieval

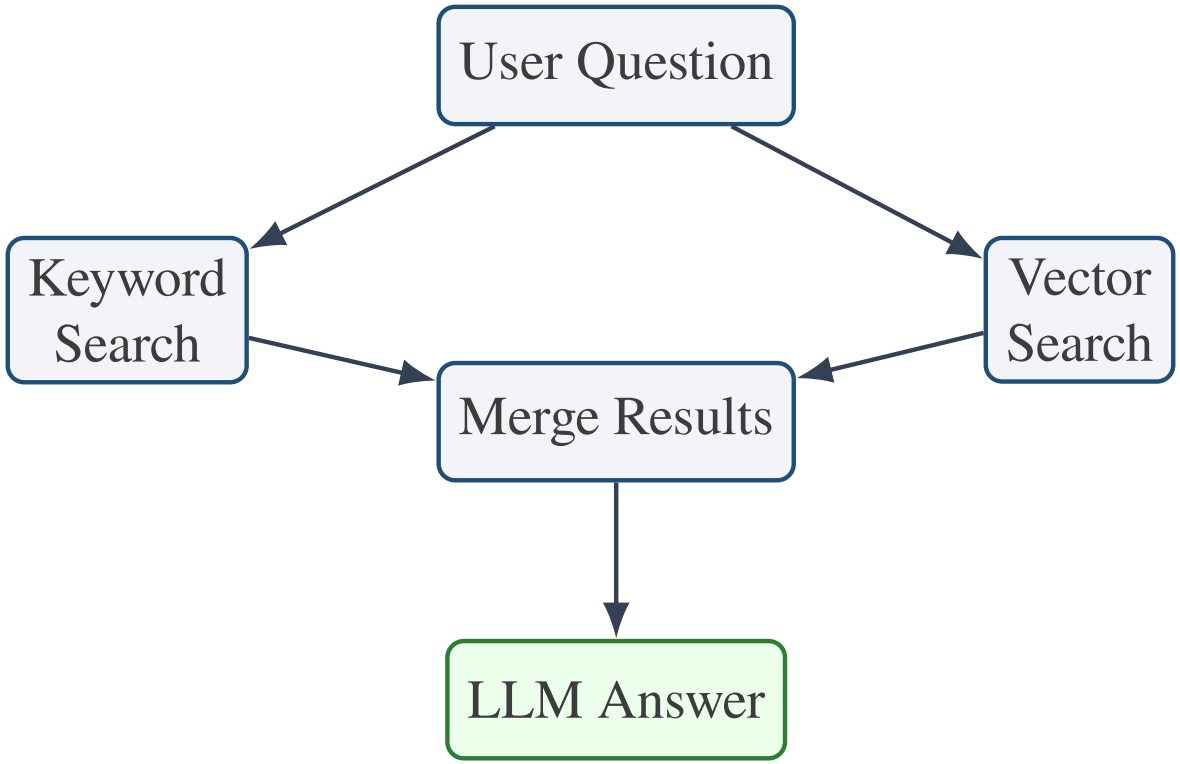
| | **Keyword** | **Vector** |
|---|---|---|
| Core idea | Match words | Match meaning |
| Strength | Exact terms | Semantic similarity |
| Weakness | Misses paraphrases | Misses exact codes |
| Good for | IDs, laws, names, clauses | Concepts, summaries, natural language |
| Failure case | “physician” vs “doctor” | “Section 12.4” |

**Teaching point**

Keyword retrieval and vector retrieval solve different problems. Neither is always best.

# 3. Hybrid RAG

**Hybrid RAG** combines keyword search and vector search.

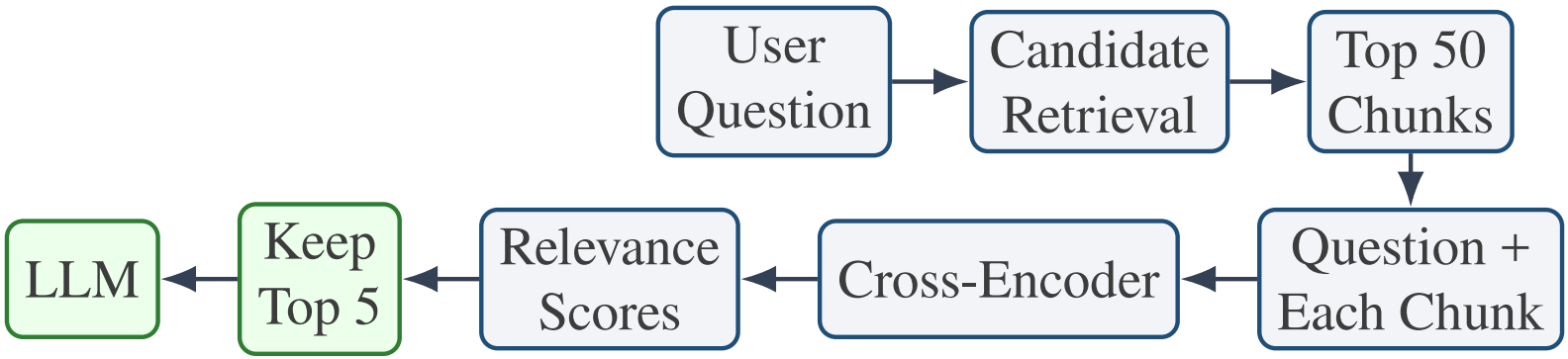



**Why it matters**

Hybrid RAG reduces blind spots by combining exact matching with semantic matching.

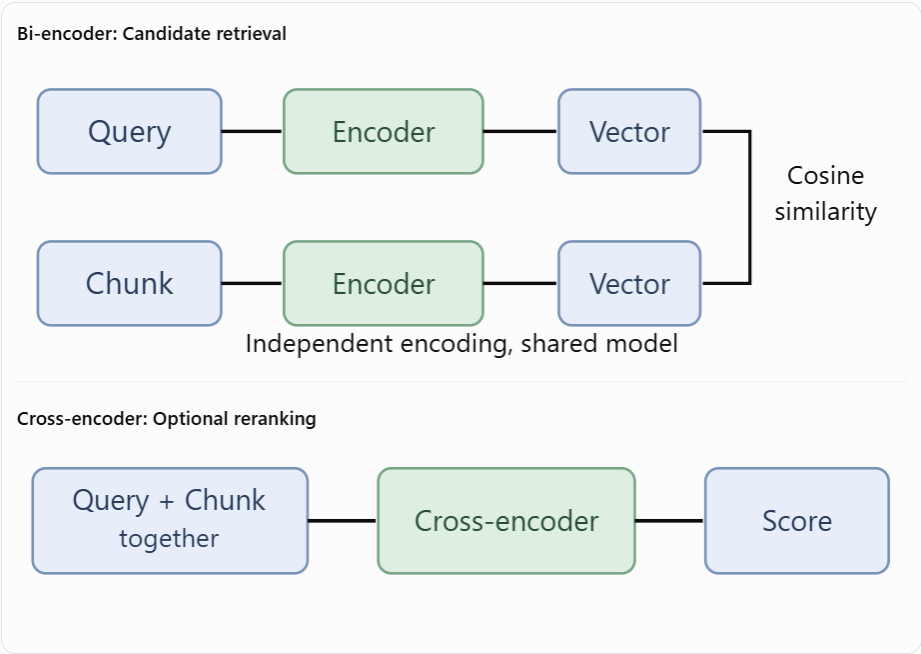
# 4. Reranked RAG

Basic RAG retrieves top-$k$ directly. Reranked RAG retrieves more, then filters better.




- **Step 1: Candidate Retrieval**
  - Retrieve a broad set of potentially relevant chunks.
  - Goal: maximize recall.
  - Use vector, keyword, or hybrid retrieval.
- **Step 2: Reranking**
  - Reevaluate candidates using a more precise model.
  - Goal: improve precision.
  - Keep the highest-ranked chunks for the LLM.

# Bi-Encoder vs. Cross-Encoder: Retrieval and Reranking




The bi-encoder processes the query and document chunks independently using a shared pretrained encoder model. It produces embeddings that can be compared using cosine similarity. Since document embeddings can be computed and stored in advance, bi-encoders enable efficient similarity search across large collections. The cross-encoder processes the query and candidate chunk together, allowing direct interaction between their tokens. This is generally more computationally expensive, which is why it is commonly used for reranking rather than searching the entire collection.

**Key Point**

Reranking is optional. It can improve relevance, but introduces additional computational cost and latency.

# Reranking Example

**Question:** What is the return deadline for damaged products?

| Rank | Retrieved chunk | Relevance |
|---|---|---|
| 1 | Customers can return unused products within 30 days. | Medium |
| 2 | Damaged products must be reported within 7 days of delivery. | High |
| 3 | Refunds are issued to the original payment method. | Low |

**Without reranking**

The system may answer "30 days."

**With reranking**

The system promotes the damaged-product rule and answers "7 days."

# Current Section

1 Retrieval-Augmented Generation (RAG)

2 Part 1: Retrieval Methods

3 Part 2: Query Strategies

4 Part 3: Reasoning Patterns

5 Part 4: Knowledge Structures

6 Part 5: Design Choices

7 Part 6: Evaluation

# 1. Query Rewriting RAG

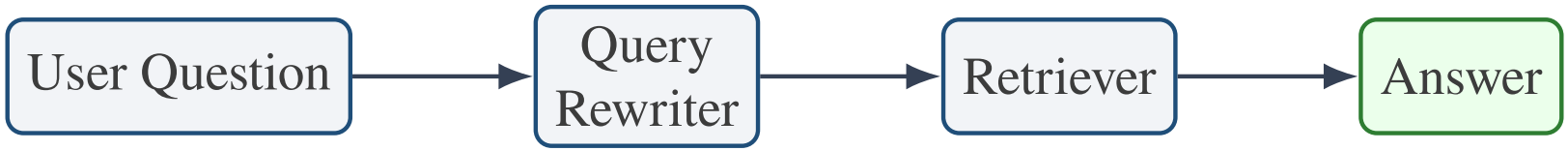
Many user questions are vague.

**Original question**

What about refunds?

**Rewritten retrieval query**

What is the company refund policy, including deadline, eligibility, exceptions, and damaged-item rules?



# 2. Multi-Query RAG

**Multi-query RAG** searches the knowledge base from several angles.

**Question:** How does the policy handle damaged returns?

Generated search queries:

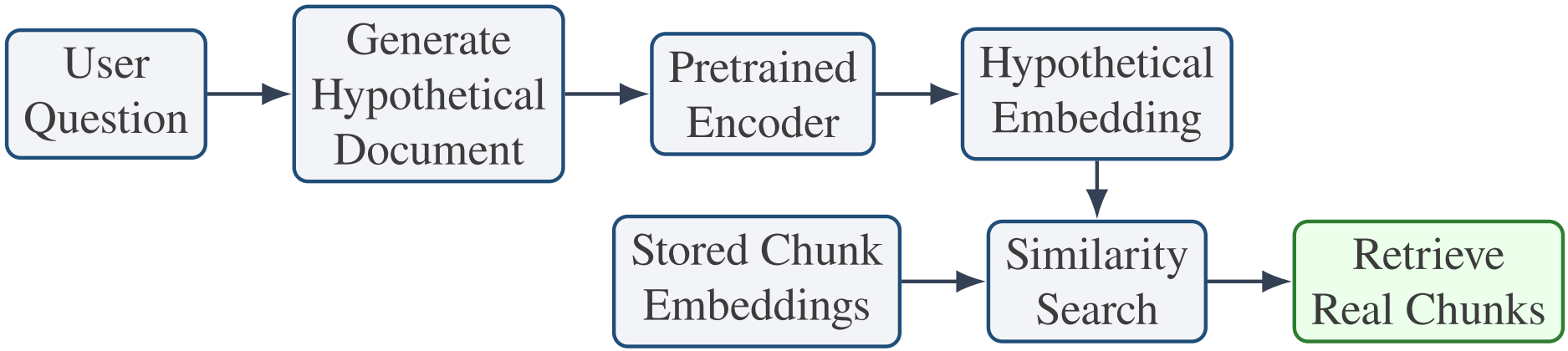
- damaged product return policy
- refund policy for damaged items
- reporting deadline for damaged delivery
- exceptions to standard return policy

**Why it helps**

A single query may miss the right wording. Multiple queries increase recall.

# 3. HyDE: Hypothetical Document Embeddings

**HyDE** creates a hypothetical answer-like document before retrieval.




**Important**

The hypothetical document may contain false details. It is used only to search, not as evidence.

**Why HyDE Helps**

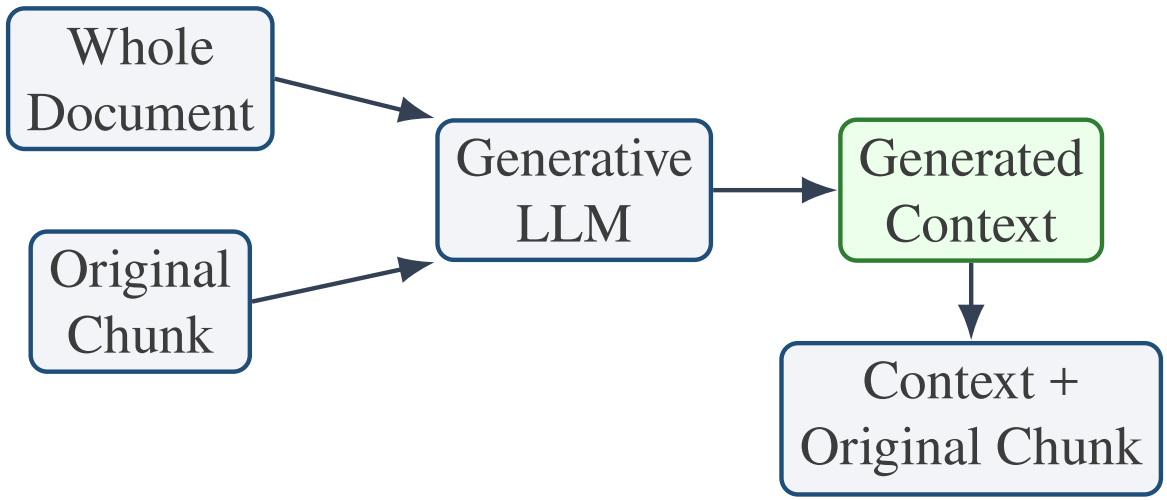
A short user query may be semantically different from the document containing the answer.
HyDE uses an LLM to generate a hypothetical answer-like document, then encodes it into an embedding.
This embedding is compared with embeddings of real document chunks to retrieve relevant evidence.

Source: Gao et al., 2023, "Precise Zero-Shot Dense Retrieval without Relevance Labels."

# 4. Contextual Retrieval

**Problem:** Individual chunks may lose important context when separated from their original document.

**Solution:** Use a generative LLM to produce a short, chunk-specific explanation based on the original document.




**Key Point**

The generated context is prepended to each chunk before creating embeddings or a BM25 index.

Source: Anthropic (2024), Introducing Contextual Retrieval.

# 5. Conversational RAG

In chat, the latest question may not stand alone.

**Conversation**

User: What is the refund policy?
User: What about damaged items?

The second question really means:

**Standalone query**

What is the refund policy for damaged items?

**How Conversational RAG Works**

1. Use chat history to understand the latest user question.
2. Reformulate it into a standalone query.
3. Retrieve relevant chunks using the reformulated query.
4. Generate an answer using the retrieved evidence and relevant conversation context.

# Current Section

1 Retrieval-Augmented Generation (RAG)

2 Part 1: Retrieval Methods

3 Part 2: Query Strategies

4 Part 3: Reasoning Patterns

5 Part 4: Knowledge Structures

6 Part 5: Design Choices

7 Part 6: Evaluation

# 1. Multi-Hop RAG

**Multi-hop RAG** performs retrieval in steps.

**Question:** Which customers affected by Policy X bought Product Y after March 2024?

Needed evidence:

1. Retrieve Policy X.
2. Identify affected customer segment.
3. Retrieve Product Y purchase records.
4. Filter by date.
5. Combine results.

**Main point**

Basic RAG does one search. Multi-hop RAG chains searches.

# Another Example of Multi-Hop RAG

**Question:**
Which university awarded the PhD to an author of *Attention Is All You Need*?

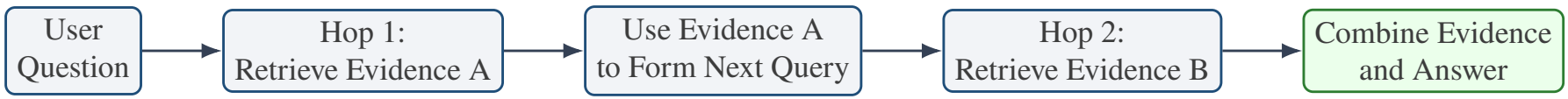
**Retrieval Steps:**

1. **Hop 1:** Retrieve the paper and identify an author, such as Ashish Vaswani.
2. **Hop 2:** Use the author's name to retrieve information about their PhD.
3. **Combine:** Use evidence from both searches to generate the answer.

**Main Point**

In multi-hop RAG, the result of one retrieval step helps determine the next retrieval query.
Basic RAG typically retrieves evidence in one step.

# Multi-Hop RAG Diagram

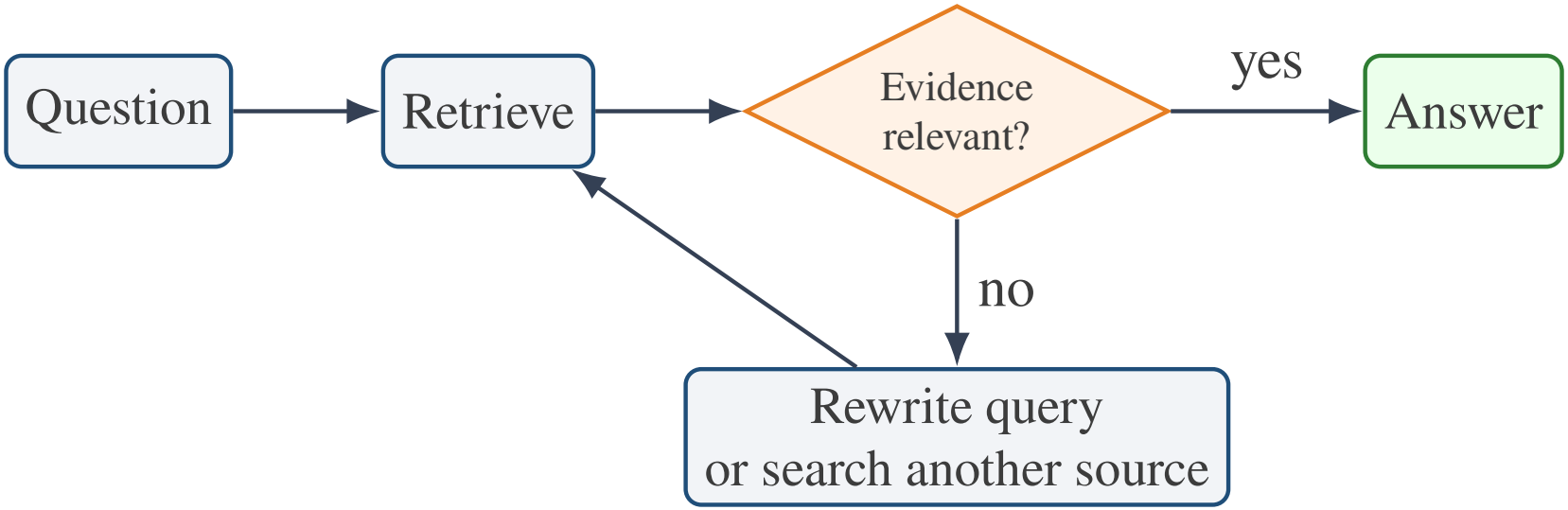



## Failure Risk

An error in the first retrieval step can affect later steps.

# 2. Corrective RAG

**Corrective RAG** checks whether retrieved evidence is good enough.




**Main point**

Do not trust retrieval blindly. Check, retry, or abstain.

# How Corrective RAG Evaluates Evidence

**Retrieval Evaluator:**
A trained evaluator model estimates how relevant each retrieved chunk is to the user question.

1. **Evaluate:** Process each (question, chunk) pair and calculate a relevance score.
2. **Assess Confidence:** Use the relevance scores to estimate the overall quality of retrieved evidence.
3. **Choose a Corrective Action:**
   - **Correct:** Refine the retrieved evidence by removing irrelevant information.
   - **Incorrect:** Discard unreliable evidence and search an external source.
   - **Ambiguous:** Combine refined evidence with external search results.
4. **Generate:** Pass the selected evidence to the LLM to generate the final answer.

**Main Point**

Corrective RAG uses an additional evaluator model to assess retrieval quality before generation.

Source: Yan et al. (2024), Corrective Retrieval Augmented Generation.

# 3. Self-RAG

**Self-RAG** makes retrieval adaptive.

The model can learn to:

- Decide whether retrieval is needed.
- Retrieve when useful.
- Critique retrieved passages.
- Reflect on its own answer.
- Improve factuality and citation behavior.

**Difference from basic RAG**

Basic RAG always retrieves a fixed number of chunks. Self-RAG retrieves and critiques more selectively.

Source: Asai et al., 2023, "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection."

# How Self-RAG Works

**Main idea:** Self-RAG trains the LLM to control retrieval and evaluate its own generation using special **reflection tokens**.

1. **Decide whether to retrieve:** The model predicts a retrieval token (`Retrieve`) to determine if external information is needed.
2. **Evaluate retrieved evidence:** The model predicts `IsREL` to assess the relevance of each passage.
3. **Generate an answer:** The model generates text using the retrieved evidence when needed.
4. **Reflect on the answer:**
   - `IsSUP`: Is the answer supported by the evidence?
   - `IsUSE`: How useful is the generated answer?
5. **Select the output:** Use reflection scores to help select the best answer.

**Key Point**

Unlike standard RAG, Self-RAG learns when to retrieve and how to critique its own output.

# 4. Agentic RAG

**Agentic RAG** lets the LLM control the retrieval workflow.

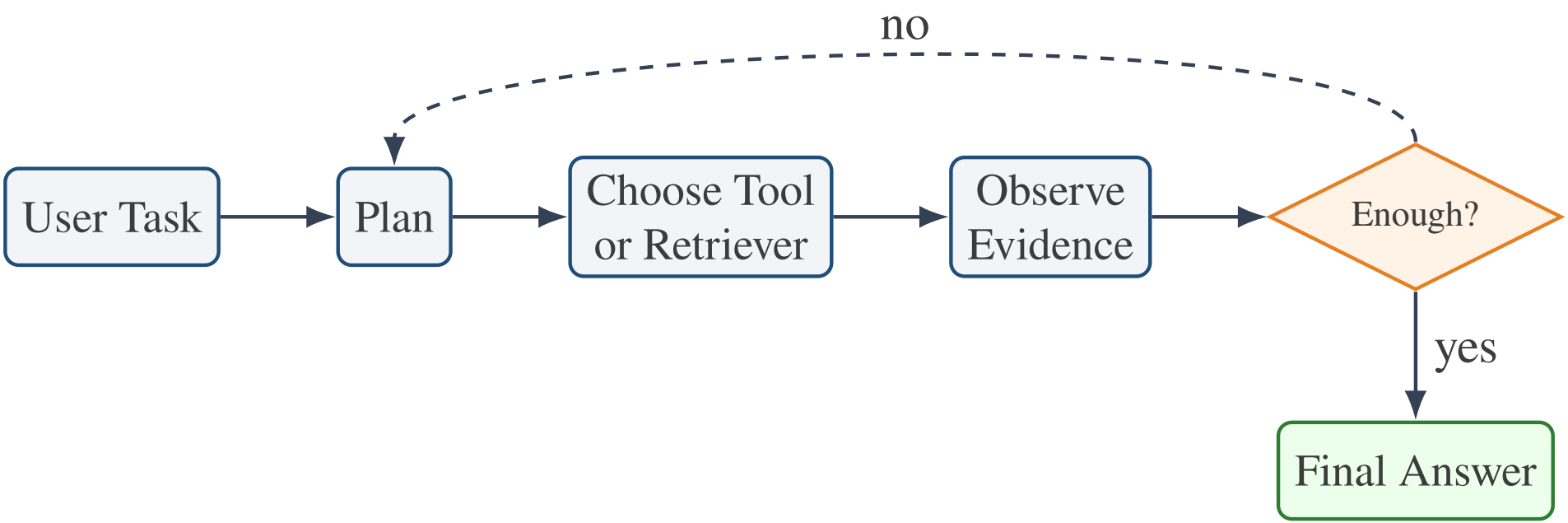
The agent can decide:

- Should I retrieve?
- Which source should I search?
- Do I need another search?
- Should I call SQL, API, web search, or a vector database?
- Is the evidence enough to answer?

**Best for**

Research assistants, customer support agents, enterprise copilots, and multi-tool workflows.

# Agentic RAG Diagram




**Risk**

More autonomy can improve answers, but also increases cost, latency, and failure modes.

# How Agentic RAG Works

**Main idea:** An LLM acts as a controller that selects tools, executes retrieval steps, and adapts its plan based on the results.

1. **Plan:** Analyze the question and identify the information needed.
2. **Select a Tool:** Choose web search, SQL, APIs, or a vector database.
3. **Execute and Observe:** The system executes the tool call and returns the results to the LLM.
4. **Evaluate:** The LLM assesses whether the retrieved evidence is sufficient.
5. **Repeat or Generate:** Search again if needed, or generate the final answer using the evidence.

**Key Point**

Agentic RAG uses an iterative plan–act–observe loop rather than a fixed retrieval pipeline.

# Current Section

1 Retrieval-Augmented Generation (RAG)

2 Part 1: Retrieval Methods

3 Part 2: Query Strategies

4 Part 3: Reasoning Patterns

5 Part 4: Knowledge Structures

6 Part 5: Design Choices

7 Part 6: Evaluation

# Understanding Knowledge Structures in RAG

**Main question:** How is knowledge organized and represented so that a RAG system can retrieve and use it?

Different types of information require different representations and retrieval approaches.

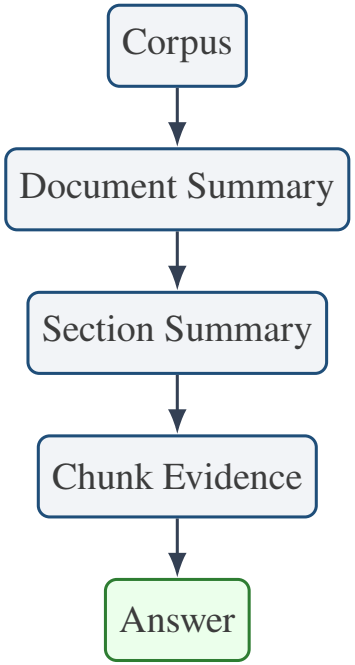
- **Hierarchical:** Organizes knowledge into documents, sections, and chunks.
- **Graph-Based:** Represents entities and their relationships.
- **Structured Data:** Organizes information into tables, records, and database fields.
- **Multimodal:** Represents information across text, images, charts, audio, and other modalities.

**Key Point**

Retrieval methods determine **how to search**. Knowledge structures determine **how information is represented and organized** for retrieval.

# 1. Hierarchical RAG

**Hierarchical RAG** retrieves evidence at multiple levels.

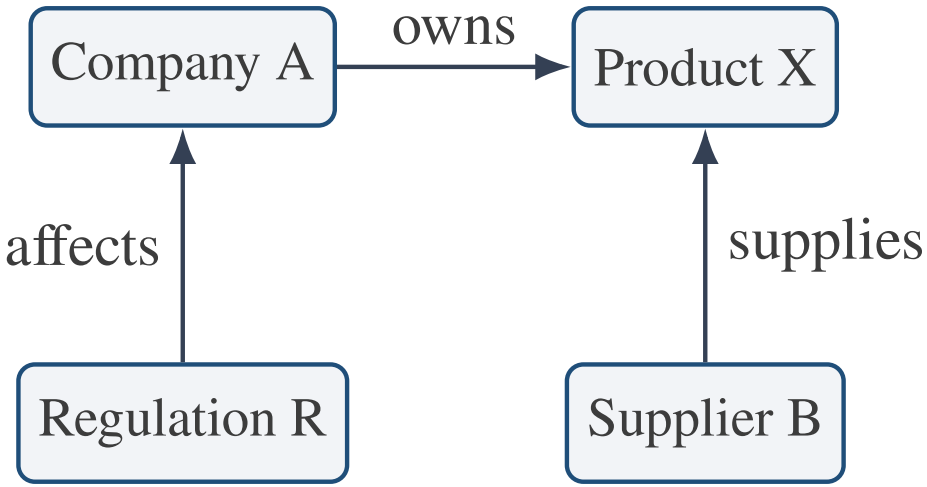



## Analogy

Find the right book, then the right chapter, then the right paragraph.

# 2. Graph RAG

**Graph RAG** represents knowledge as entities and relationships.




**Best for**

Relationship-heavy questions across many documents.

Microsoft describes GraphRAG as combining text extraction, network analysis, and LLM prompting over graph structures.

# Graph RAG vs. Chunk RAG

| | **Chunk RAG** | **Graph RAG** |
|---|---|---|
| Unit of retrieval | Text chunks | Entities and relations |
| Good for | Direct answers | Relationship reasoning |
| Question type | "What does this policy say?" | "Who is connected to whom?" |
| Weakness | Misses global structure | More complex to build |

**Teaching point**

Chunk RAG retrieves passages. Graph RAG retrieves structured relationships.

# 3. Structured-Data RAG

**Structured-data RAG** connects the LLM to databases, APIs, and tables.

Example question:

What were our top 5 products by revenue last quarter?

A text chunk is not enough. The system should query data.



# 4. Multimodal RAG

**Multimodal RAG** retrieves and reasons over non-text sources.

Possible sources:

- Text documents
- Tables
- Charts
- Images
- Slides
- Scanned PDFs
- Audio or video transcripts

**Example**

Question: What trend is shown in the Q3 revenue chart?

The answer requires reading the chart, not only text.

# Current Section

1 Retrieval-Augmented Generation (RAG)

2 Part 1: Retrieval Methods

3 Part 2: Query Strategies

4 Part 3: Reasoning Patterns

5 Part 4: Knowledge Structures

6 Part 5: Design Choices

7 Part 6: Evaluation

# Understanding Design Choices in RAG

**Main question:**
How should we configure and optimize a RAG system to improve retrieval quality and answer generation?

**Important Design Decisions:**

- **Chunking:** How large should chunks be? Should they overlap?
- **Indexing:** How should knowledge be indexed for efficient retrieval?
- **Contextual Retrieval:** Should we add document-level context to individual chunks?
- **Metadata Filtering:** Which documents should be eligible for retrieval?
- **Prompting:** How should the LLM use retrieved evidence to generate reliable answers?

**Key Point**

Design choices affect retrieval accuracy, answer quality, computational cost, and latency.

# Chunking Strategies

Chunking is not just splitting text every 500 tokens.

Common strategies:

- Fixed-size chunks
- Overlapping chunks
- Sentence or paragraph chunks
- Section-aware chunks
- Parent-child chunks
- Table-aware chunks

**Rule of thumb**

Chunks should be small enough to be focused, but large enough to preserve meaning.

# Indexing Strategies

A production RAG system may store more than one index.

- Dense vector index
- Keyword index
- Metadata index
- Document summary index
- Entity graph index
- Table or SQL index

Indexing strategy decides how to organize the data so that retrieval method can search efficiently.

**Main point**

The index should match the type of question and the type of knowledge.

# Metadata Filtering

RAG often improves when retrieval uses metadata.

Metadata examples:

- Date
- Author
- Document type
- Department
- Access permission
- Version
- Region

**Example**

Search only HR policy documents published after January 2025.

# Prompting the Generator

After retrieval, the LLM still needs good instructions.

Useful rules:

- Answer only from the retrieved evidence.
- Cite sources when possible.
- Say “I do not know” when evidence is missing.
- Do not merge unrelated chunks.
- Explain uncertainty when sources conflict.

**Bad RAG behavior**

The model retrieves evidence but answers from memory anyway.

# Current Section

1 Retrieval-Augmented Generation (RAG)

2 Part 1: Retrieval Methods

3 Part 2: Query Strategies

4 Part 3: Reasoning Patterns

5 Part 4: Knowledge Structures

6 Part 5: Design Choices

7 Part 6: Evaluation

# RAG Evaluation Has Two Layers

## Retrieval evaluation

- Did we retrieve the right evidence?
- Is the gold chunk in top-$k$?
- How much irrelevant text was retrieved?

## Generation evaluation

- Is the answer correct?
- Is it grounded in evidence?
- Are citations accurate?

## Core idea

A good answer needs good retrieval and good generation.

# Common RAG Metrics

| Metric | Question it answers | Category |
|---|---|---|
| Recall@k | Was the correct evidence in the top-$k$? | Retrieval |
| Precision@k | How much retrieved evidence was relevant? | Retrieval |
| MRR | How high was the first correct result ranked? | Retrieval |
| NDCG@k | Are relevant chunks ranked near the top? | Retrieval |
| Faithfulness | Is the answer supported by the retrieved evidence? | Generation |
| Answer correctness | Is the final answer right? | Generation |
| Citation accuracy | Do citations point to the right source? | Generation |
| Latency | How long does the system take? | Both |
| Cost | How expensive is each answer? | Both |

# Information Retrieval: Modern Neural IR

If you remember Information Retrieval from NLP 1, we compared **classical IR** with **modern neural IR**.

**Main Idea**

Modern neural IR retrieves information by meaning, not only by exact keyword matching.

A neural model reads the query and the document, then converts them into dense vector representations called **embeddings**.

If the query vector and document vector are close in the embedding space, the document is considered relevant.

**Query** $\rightarrow$ **Embedding** **Document** $\rightarrow$ **Embedding**

Closer vectors $\Rightarrow$ more relevant results

# RAG: Retrieval + Generation

Now we can see how neural retrieval becomes part of RAG:

$$\text{RAG} = \text{Retrieval} + \text{Generation}$$

**Main Idea**

RAG first retrieves relevant information, then uses a language model to generate an answer from it.

This connects neural IR with modern AI systems.

Some search engines use RAG-like systems when they generate AI summaries from retrieved web sources.

# Very Simple Demo

# Code — Documents

```
documents = [
    {
        "id": "doc1",
        "title": "Refund Policy",
        "text": """
        Customers can request a refund within 30 days of purchase.
        The item must be unused and returned in its original packaging.
        Refunds are issued to the original payment method.
        """
    },
    {
        "id": "doc2",
        "title": "Damaged Products",
        "text": """
        Damaged products must be reported within 7 days of delivery.
        Customers should send a photo of the damaged item.
        After approval, the company will offer a replacement or refund.
        """
    },
    {
        "id": "doc3",
        "title": "Shipping Policy",
        "text": """
        Standard shipping takes 3 to 5 business days.
        Express shipping takes 1 to 2 business days.
        International shipping may take longer.
        """
    },
    {
        "id": "doc4",
        "title": "Section 12.4 Termination",
        "text": """
        Section 12.4 states that either party may terminate the agreement
        with 30 days written notice if the other party violates the contract.
        """
    },
    {
        "id": "doc5",
        "title": "Medical Synonyms Example",
        "text": """
        A physician must review the patient record before treatment.
        The doctor should confirm allergies and medication history.
        """
    }
]
```

# Code—Imports

```
import re
import numpy as np
from rank_bm25 import BM25Okapi
from sentence_transformers import SentenceTransformer, CrossEncoder
from sklearn.metrics.pairwise import cosine_similarity
```

# Code—Chunking

```
# ------------------------------------------------------------
# Method 1: Chunking
# ------------------------------------------------------------

def chunk_text(text, chunk_size=35, overlap=8):
    """
    Splits a document into smaller chunks.

    This is for the Chunking method.
    In real RAG, chunks are often 300 to 800 tokens.
    Here we use small chunks for teaching.
    """

    words = text.split()
    chunks = []

    start = 0
    while start < len(words):
        end = start + chunk_size
        chunk = " ".join(words[start:end])
        chunks.append(chunk)

        start = end - overlap

        if start >= len(words):
            break

    return chunks
```

# Chunking—Continued

This function takes the original documents and turns them into searchable chunks.

```python
def build_chunks(documents):
    """
    Creates chunks with metadata.
    Metadata helps us know where each chunk came from.
    """

    chunks = []

    for doc in documents:
        doc_chunks = chunk_text(doc["text"])

        for i, chunk in enumerate(doc_chunks):
            chunks.append({
                "chunk_id": f"{doc['id']}_chunk_{i}",
                "doc_id": doc["id"],
                "title": doc["title"],
                "text": chunk
            })

    return chunks


chunks = build_chunks(documents)
```

# Code—Loading Pretrained Transformer-based Models

We use two pretrained Transformer-based models for retrieval and reranking.

- **Embedding Model (Bi-Encoder):**
  - `all-MiniLM-L6-v2`
  - Generates embeddings for semantic retrieval.
  - Encodes queries and chunks independently.

- **Reranking Model (Cross-Encoder):**
  - `ms-marco-MiniLM-L-6-v2`
  - Scores query-chunk pairs for reranking.
  - Processes the query and chunk together.

```
# ------------------------------------------------------------
# Load embedding and reranker models
# ------------------------------------------------------------

embedding_model = SentenceTransformer("all-MiniLM-L6-v2")

# CrossEncoder is used for reranking.
# It scores a query and a chunk together.
reranker_model = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")
```

# Code—Embedding

```
# ------------------------------------------------------------
# Method 2: Embeddings
# ------------------------------------------------------------

def create_embeddings(chunks):
    """
    Converts text chunks into dense vectors.

    This is for the Embeddings method.
    """

    texts = [chunk["text"] for chunk in chunks]
    embeddings = embedding_model.encode(texts, convert_to_numpy=True)
    return embeddings


chunk_embeddings = create_embeddings(chunks)
```

# Code—Vector RAG

```python
# ------------------------------------------------------------
# Method 3: Vector RAG
# ------------------------------------------------------------

def vector_search(question, chunks, chunk_embeddings, k=3):
    """
    Retrieves chunks using semantic similarity.

    This is for Vector RAG.
    It compares the question embedding with chunk embeddings.
    """

    question_embedding = embedding_model.encode([question], convert_to_numpy=True)

    scores = cosine_similarity(question_embedding, chunk_embeddings)[0]

    ranked_indices = np.argsort(scores)[::-1][:k]

    results = []
    for idx in ranked_indices:
        result = chunks[idx].copy()
        result["score"] = float(scores[idx])
        result["method"] = "Vector RAG"
        results.append(result)

    return results
```

# Code—Keyword RAG, BM25

```
# ------------------------------------------------------------
# Method 4: Keyword RAG, BM25
# ------------------------------------------------------------

def tokenize(text):
    """
    Simple tokenizer for BM25 keyword search.
    """
    text = text.lower()
    return re.findall(r"\b\w+\.?\w*\b", text)

tokenized_chunks = [tokenize(chunk["text"]) for chunk in chunks]
bm25 = BM25Okapi(tokenized_chunks)


def keyword_search(question, chunks, k=3):
    """
    Retrieves chunks using BM25 keyword matching.
    This is for Keyword RAG.
    It works well for exact terms, codes, names, and section numbers.
    """
    query_tokens = tokenize(question)
    scores = bm25.get_scores(query_tokens)
    ranked_indices = np.argsort(scores)[::-1][:k]

    results = []
    for idx in ranked_indices:
        result = chunks[idx].copy()
        result["score"] = float(scores[idx])
        result["method"] = "Keyword RAG, BM25"
        results.append(result)

    return results
```

# Code—Hybrid RAG

```
# ------------------------------------------------------------
# Method 5: Hybrid RAG
# ------------------------------------------------------------

def normalize_scores(scores):
    """
    Normalizes scores into the range 0 to 1.
    """

    scores = np.array(scores, dtype=float)

    if scores.max() == scores.min():
        return np.zeros_like(scores)

    return (scores - scores.min()) / (scores.max() - scores.min())
```

# Hybrid RAG—Continued

```python
def hybrid_search(question, chunks, chunk_embeddings, k=3, alpha=0.5):
    """
    Combines vector search and BM25 keyword search.

    This is for Hybrid RAG.
    alpha controls the balance:
    alpha = 0.5 means half vector score, half keyword score.
    """

    question_embedding = embedding_model.encode([question], convert_to_numpy=True)

    vector_scores = cosine_similarity(question_embedding, chunk_embeddings)[0]
    keyword_scores = bm25.get_scores(tokenize(question))

    vector_scores_norm = normalize_scores(vector_scores)
    keyword_scores_norm = normalize_scores(keyword_scores)

    hybrid_scores = alpha * vector_scores_norm + (1 - alpha) * keyword_scores_norm

    ranked_indices = np.argsort(hybrid_scores)[::-1][:k]

    results = []
    for idx in ranked_indices:
        result = chunks[idx].copy()
        result["score"] = float(hybrid_scores[idx])
        result["method"] = "Hybrid RAG"
        results.append(result)

    return results
```

# Code—Reranked RAG

```python
# ------------------------------------------------------------
# Method 6: Reranked RAG
# ------------------------------------------------------------

def reranked_search(question, chunks, chunk_embeddings, retrieve_k=5, final_k=3):
    """
    First retrieves many chunks with vector search.
    Then reranks them with a cross-encoder.

    This is for Reranked RAG.
    Retriever is fast.
    Reranker is slower but more precise.
    """

    initial_results = vector_search(question, chunks, chunk_embeddings, k=retrieve_k)

    pairs = [(question, result["text"]) for result in initial_results]

    rerank_scores = reranker_model.predict(pairs)

    for result, score in zip(initial_results, rerank_scores):
        result["score"] = float(score)
        result["method"] = "Reranked RAG"

    reranked_results = sorted(
        initial_results,
        key=lambda x: x["score"],
        reverse=True
    )

    return reranked_results[:final_k]
```

# Code—Helper

```python
# ------------------------------------------------------------
# Helper function to print results
# ------------------------------------------------------------

def print_results(title, question, results):
    print("\n" + "=" * 80)
    print(title)
    print("=" * 80)
    print(f"Question: {question}\n")

    for i, result in enumerate(results, start=1):
        print(f"Rank {i}")
        print(f"Method: {result['method']}")
        print(f"Title: {result['title']}")
        print(f"Score: {result['score']:.4f}")
        print(f"Text: {result['text']}")
        print("-" * 80)
```

# Result—Vector RAG

```
question_1 = "What is the return deadline for damaged products?"

# Vector RAG
vector_results = vector_search(question_1, chunks, chunk_embeddings, k=3)

print_results("Method 3: Vector RAG", question_1, vector_results)
```

```
================================================================================
Method 3: Vector RAG
================================================================================
Question: What is the return deadline for damaged products?

Rank 1
Method: Vector RAG
Title: Damaged Products
Score: 0.6946
Text: Damaged products must be reported within 7 days of delivery. Customers should send a photo of the damaged item.
--------------------------------------------------------------------------------
Rank 2
Method: Vector RAG
Title: Refund Policy
Score: 0.5314
Text: Customers can request a refund within 30 days of purchase. The item must be unused and returned in its original
--------------------------------------------------------------------------------
Rank 3
Method: Vector RAG
Title: Damaged Products
Score: 0.3809
Text: or refund.
--------------------------------------------------------------------------------
```

# Result—Keyword RAG, BM25

```
# Keyword RAG, BM25
keyword_results = keyword_search(question_1, chunks, k=3)
print_results("Method 4: Keyword RAG, BM25", question_1, keyword_results)
```

```
...
================================================================================
Method 4: Keyword RAG, BM25
================================================================================
Question: What is the return deadline for damaged products?

Rank 1
Method: Keyword RAG, BM25
Title: Damaged Products
Score: 3.2292
Text: Damaged products must be reported within 7 days of delivery. Customers should send a photo of the
--------------------------------------------------------------------------------
Rank 2
Method: Keyword RAG, BM25
Title: Section 12.4 Termination
Score: 0.4967
Text: Section 12.4 states that either party may terminate the agreement with 30 days written notice if
--------------------------------------------------------------------------------
Rank 3
Method: Keyword RAG, BM25
Title: Medical Synonyms Example
Score: 0.4572
Text: A physician must review the patient record before treatment. The doctor should confirm allergies
--------------------------------------------------------------------------------
```

# Result—Hybrid RAG

```
# Hybrid RAG
hybrid_results = hybrid_search(question_1, chunks, chunk_embeddings, k=3, alpha=0.5)
print_results("Method 5: Hybrid RAG", question_1, hybrid_results)
```

```
...

================================================================================
Method 5: Hybrid RAG
================================================================================
Question: What is the return deadline for damaged products?

Rank 1
Method: Hybrid RAG
Title: Damaged Products
Score: 1.0000
Text: Damaged products must be reported within 7 days of delivery. Customers should send a photo of th
--------------------------------------------------------------------------------
Rank 2
Method: Hybrid RAG
Title: Refund Policy
Score: 0.4289
Text: Customers can request a refund within 30 days of purchase. The item must be unused and returned
--------------------------------------------------------------------------------
Rank 3
Method: Hybrid RAG
Title: Section 12.4 Termination
Score: 0.2983
Text: Section 12.4 states that either party may terminate the agreement with 30 days written notice if
--------------------------------------------------------------------------------
```

# Result—Reranked RAG

```
# Reranked RAG
reranked_results = reranked_search(question_1, chunks, chunk_embeddings, retrieve_k=5, final_k=3)
print_results("Method 6: Reranked RAG", question_1, reranked_results)
```

```
...
================================================================================
Method 6: Reranked RAG
================================================================================
Question: What is the return deadline for damaged products?

Rank 1
Method: Reranked RAG
Title: Damaged Products
Score: 4.6961
Text: Damaged products must be reported within 7 days of delivery. Customers should send a photo of
--------------------------------------------------------------------------------
Rank 2
Method: Reranked RAG
Title: Refund Policy
Score: -2.3868
Text: Customers can request a refund within 30 days of purchase. The item must be unused and returne
--------------------------------------------------------------------------------
Rank 3
Method: Reranked RAG
Title: Damaged Products
Score: -8.5661
Text: or refund.
--------------------------------------------------------------------------------
```

# Comparing RAG Retrieval Methods Using Haystack Components

**Retrieval Methods:**

- **Vector Retrieval:** Uses pretrained MiniLM embeddings and semantic similarity.
- **Keyword Retrieval (BM25):** Ranks documents based on keyword matching.
- **Hybrid Retrieval:** Combines vector and BM25 rankings using Reciprocal Rank Fusion (RRF).
- **Reranked Retrieval:** Uses a pretrained cross-encoder to rerank retrieved candidates.

**Implementation**

Uses ready-made Haystack components rather than implementing retrieval algorithms from scratch.

# Same Documents

```python
# Create the document collection
!pip -q install -U haystack-ai sentence-transformers-haystack

from haystack import Document
from haystack.document_stores.in_memory import InMemoryDocumentStore

policies = [
    ("doc1", "Refund Policy",
     "Customers can request a refund within 30 days of purchase. "
     "The item must be unused and in its original packaging. "
     "Refunds go to the original payment method."),
    ("doc2", "Damaged Products",
     "Damaged products must be reported within 7 days of delivery. "
     "Customers should send a photo of the damaged item. "
     "After approval, the company offers a replacement or refund."),
    ("doc3", "Shipping Policy",
     "Standard shipping takes 3 to 5 business days. "
     "Express shipping takes 1 to 2 business days. International shipping may take longer."),
    ("doc4", "Section 12.4 Termination",
     "Section 12.4 states that either party may terminate the agreement "
     "with 30 days written notice if the other party violates the contract."),
    ("doc5", "Medical Record Review",
     "A physician must review the patient record before treatment. "
     "The doctor should confirm allergies and medication history."),
]

documents = [Document(id=id, content=text, meta={"title": title})
             for id, title, text in policies]

store = InMemoryDocumentStore(embedding_similarity_function="cosine")
```

# Load Pretrained Models

```python
# Load pretrained models and index documents

from haystack.components.retrievers.in_memory import (
    InMemoryBM25Retriever, InMemoryEmbeddingRetriever)
from haystack.components.joiners import DocumentJoiner
from haystack_integrations.components.embedders.sentence_transformers import (
    SentenceTransformersDocumentEmbedder, SentenceTransformersTextEmbedder)
from haystack_integrations.components.rankers.sentence_transformers import (
    SentenceTransformersSimilarityRanker)

MODEL = "sentence-transformers/all-MiniLM-L6-v2"

doc_embedder = SentenceTransformersDocumentEmbedder(model=MODEL)
doc_embedder.warm_up()
store.write_documents(doc_embedder.run(documents)["documents"])

query_embedder = SentenceTransformersTextEmbedder(model=MODEL)
query_embedder.warm_up()

vector_retriever = InMemoryEmbeddingRetriever(document_store=store)
bm25_retriever = InMemoryBM25Retriever(document_store=store)
hybrid_joiner = DocumentJoiner(join_mode="reciprocal_rank_fusion")

reranker = SentenceTransformersSimilarityRanker(
    model="cross-encoder/ms-marco-MiniLM-L-6-v2"
)
reranker.warm_up()
```

# Compare Methods

```python
# Compare four methods

question = "What is the return deadline for damaged products?"
query_embedding = query_embedder.run(text=question)["embedding"]

# 1. Vector retrieval
vector_results = vector_retriever.run(query_embedding=query_embedding, top_k=3)["documents"]

# 2. Keyword retrieval (BM25)
bm25_results = bm25_retriever.run(query=question, top_k=3)["documents"]

# 3. Hybrid retrieval (BM25 + vector, using built-in rank fusion)
hybrid_results = hybrid_joiner.run(documents=[
    bm25_retriever.run(query=question, top_k=5)["documents"],
    vector_retriever.run(query_embedding=query_embedding, top_k=5)["documents"],
])["documents"][:3]

# 4. Reranked retrieval (vector candidates + pretrained cross-encoder)
candidates = vector_retriever.run(query_embedding=query_embedding, top_k=5)["documents"]
reranked_results = reranker.run(query=question, documents=candidates, top_k=3)["documents"]

# Display the top results from each method
for method, results in {
    "Vector RAG": vector_results,
    "Keyword RAG (BM25)": bm25_results,
    "Hybrid RAG": hybrid_results,
    "Reranked RAG": reranked_results,
}.items():
    print(f"\n{method}")
    for rank, doc in enumerate(results, start=1):
        print(f"  {rank}. {doc.meta['title']}  (score={doc.score:.3f})")
        print(f"     {doc.content}")
```

# Result

```
Vector RAG
  1. Damaged Products  (score=0.697)
     Damaged products must be reported within 7 days of delivery. Customers should send a photo of the damaged item. After approval, the company offers a repla
  2. Refund Policy  (score=0.524)
     Customers can request a refund within 30 days of purchase. The item must be unused and in its original packaging. Refunds go to the original payment metho
  3. Section 12.4 Termination  (score=0.342)
     Section 12.4 states that either party may terminate the agreement with 30 days written notice if the other party violates the contract.

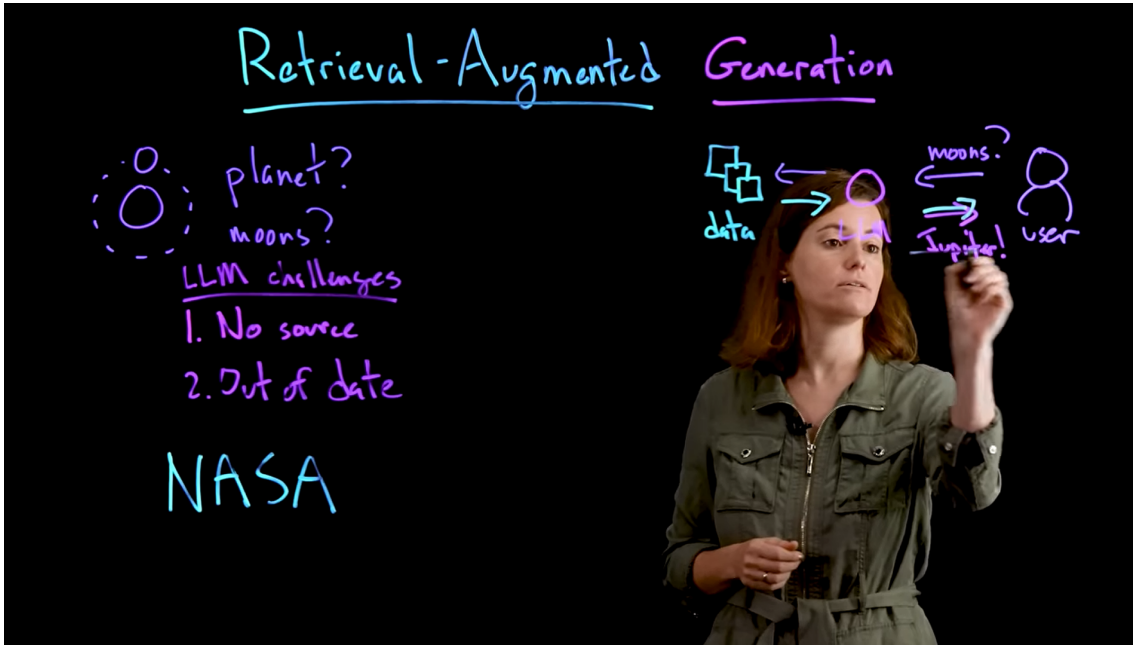
Keyword RAG (BM25)
  1. Damaged Products  (score=4.154)
     Damaged products must be reported within 7 days of delivery. Customers should send a photo of the damaged item. After approval, the company offers a repla
  2. Section 12.4 Termination  (score=0.504)
     Section 12.4 states that either party may terminate the agreement with 30 days written notice if the other party violates the contract.
  3. Medical Record Review  (score=0.480)
     A physician must review the patient record before treatment. The doctor should confirm allergies and medication history.

Hybrid RAG
  1. Damaged Products  (score=1.000)
     Damaged products must be reported within 7 days of delivery. Customers should send a photo of the damaged item. After approval, the company offers a repla
  2. Section 12.4 Termination  (score=0.976)
     Section 12.4 states that either party may terminate the agreement with 30 days written notice if the other party violates the contract.
  3. Refund Policy  (score=0.968)
     Customers can request a refund within 30 days of purchase. The item must be unused and in its original packaging. Refunds go to the original payment metho

Reranked RAG
  1. Damaged Products  (score=0.990)
     Damaged products must be reported within 7 days of delivery. Customers should send a photo of the damaged item. After approval, the company offers a repla
  2. Refund Policy  (score=0.009)
     Customers can request a refund within 30 days of purchase. The item must be unused and in its original packaging. Refunds go to the original payment metho
  3. Section 12.4 Termination  (score=0.000)
     Section 12.4 states that either party may terminate the agreement with 30 days written notice if the other party violates the contract.
```

# YouTube Video

## Watch on YouTube



# Congratulations!

Now you understand the popular RAG framework!

